

Universitatea Tehnica din Cluj-Napoca
Departament Calculatoare

Programming Techniques in Java

UML Modeling, Architectural Patterns

T. Cioara, V. Chifu, C. Pop, D. Mitrea
2026

Models



What is a model?

View the design of the application

Simplification of reality

Start of coding

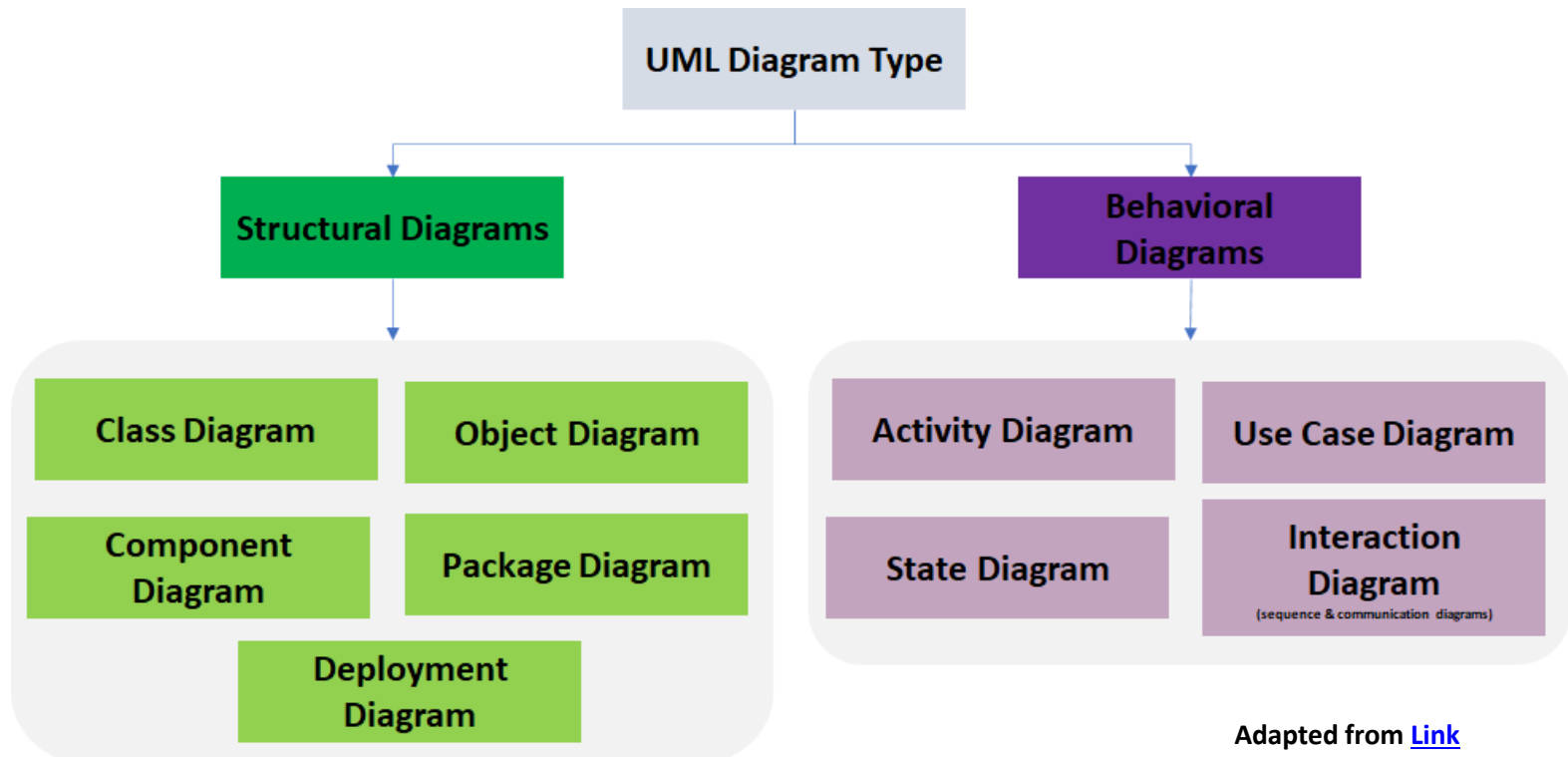


What are models for?

- Checking the requirements and domain knowledge
- Detail the design of a system
- Exploring multiple solutions (e.g., cost, time, etc)
- Guiding the development process
- Making changes and reducing dependencies

UML Modeling

- Unified Modelling Language
 - visualizing, specifying, constructing, and documenting
 - artifacts of a software system



Adapted from [Link](#)

UML Modeling - Use Case Diagrams

- Use-cases
 - Set of scenarios related to how the system is used
 - Help to discover
 - System entities
 - System actors (roles)
 - Attributes
 - Behavior
 - How actors interact with system resources
- Use case diagrams
 - Graphically capture system actors, use-cases and their relationships => requirements
- Use-case document
 - Describes the sequence of actor - system interaction

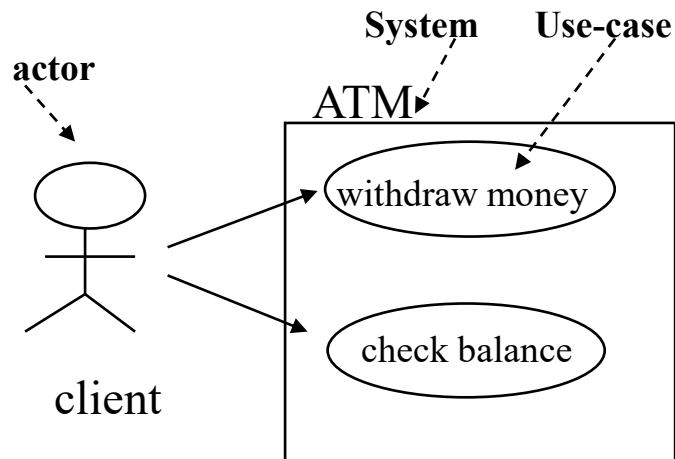
UML Modeling - Use Case Diagrams

- Use-Cases Document Example

Use Case <S#. #>: <Use Case Name (best use a verb of action)>	
Brief Description	<A brief description of the use case, no longer than a small paragraph; preferably keep it one clear goal per use case>
Parent Scenario	<Scenario number>
Actor(s)	<All users interacting with the System; preferably identified by role name>
Priority	<Depending on how many use cases we end up with we may need to prioritize them based on importance to meeting the project's objective: high (must do) medium (should do, time permitted) low (nice to do, but most likely not)>
Trigger	<Concrete actions by users interacting with the System to initiate use case>
Pre-conditions	<System State prior to launching the use case >
Post-conditions	<System State expected after the use case has been completed >
Basic Flow	<Describe the basic flow of events during use case Step 1. Step 2. ... Step N. >
Alternate Flow(s)	<If applicable, provide the steps of less common user/system interactions>
Exception Flow(s)	< Anything that may happen that would prevent the user from achieving their goal >

UML Modeling - Use Case Diagrams

- Example - ATM



- Use case description example

Use Case: withdraw money

Primary Actor: Client

Success Scenario Steps:

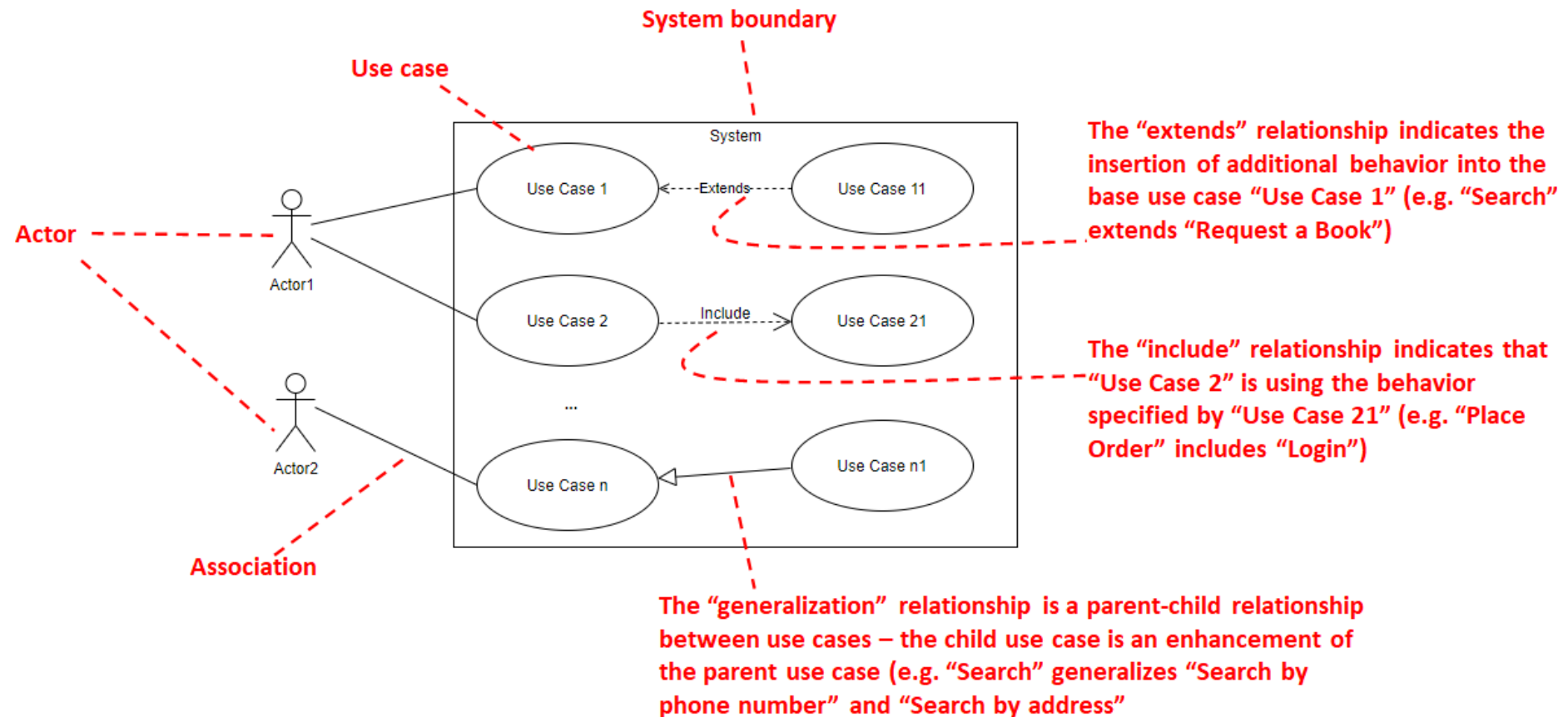
1. The client introduces the card in the ATM
2. The ATM checks if the card is valid
3. The ATM requests the client to insert the pin
4. The client introduces the pin
5. The ATM checks if the date is correct
6. The ATM requests the client to specify the amount of money
7. The client introduces the amount of money
8. The ATM checks if the amount of money is under the daily limit
9. The ATM asks the client if he wants a receipt
10. The client specifies that he wants the receipt
11. The ATM returns the card
12. The client takes the card
13. The ATM returns the money and the receipt
14. The client takes the money and the receipt

Alternative Sequences:

- a) Incorrect pin
 - The ATM tells the client that the pin is incorrect
 - The scenario returns to step 3
- b) The amount of money is higher than the daily limit
 - The ATM tells the client that the amount of money is higher than the daily limit
 - The scenario returns to step 7

UML Modeling - Use Case Diagrams

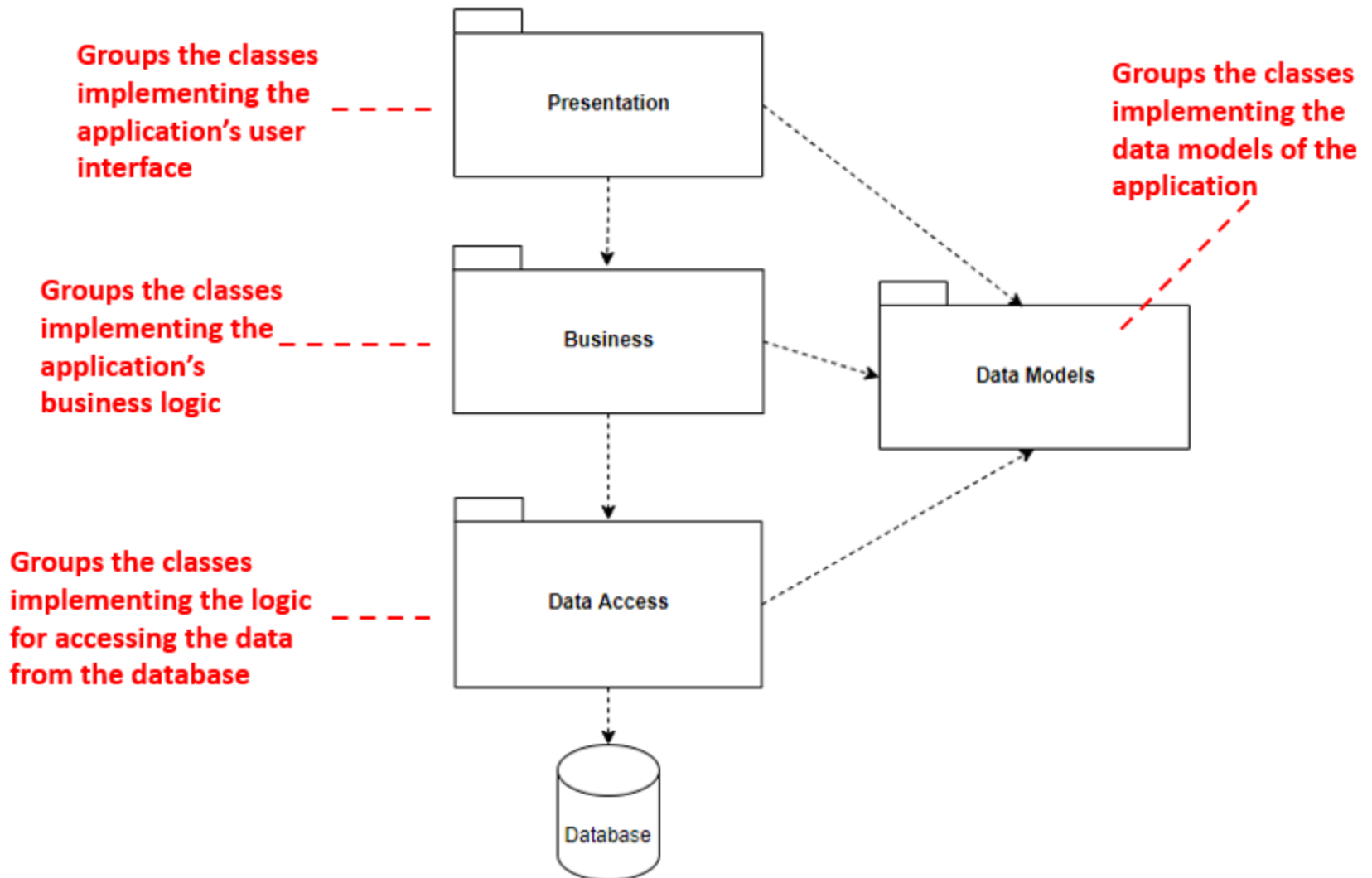
- Graphical representation



UML Modeling - Package Diagrams

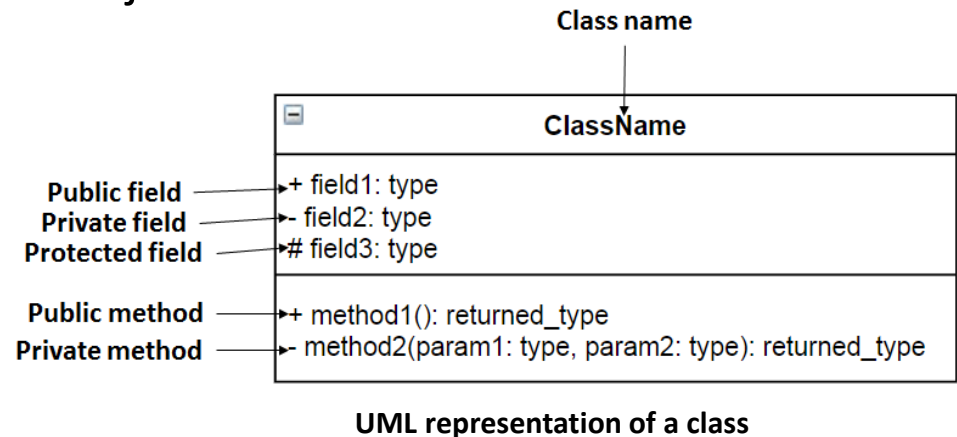
- Illustrate packages of classes and their dependencies
- Useful in decomposing large systems into subsystems
- UML package
 - Collection of modelling elements that are grouped together because they are logically related
- Should apply the principles of high cohesion and low coupling

UML Modeling - Package Diagrams



UML Modeling - Class and Object Diagrams

- Illustrate the logical structure of the system
 - Classes (attributes, operations) and objects
 - Relationships
 - associations
 - aggregation/ composition
 - generalization (inheritance)
 - realization
 - Dependency

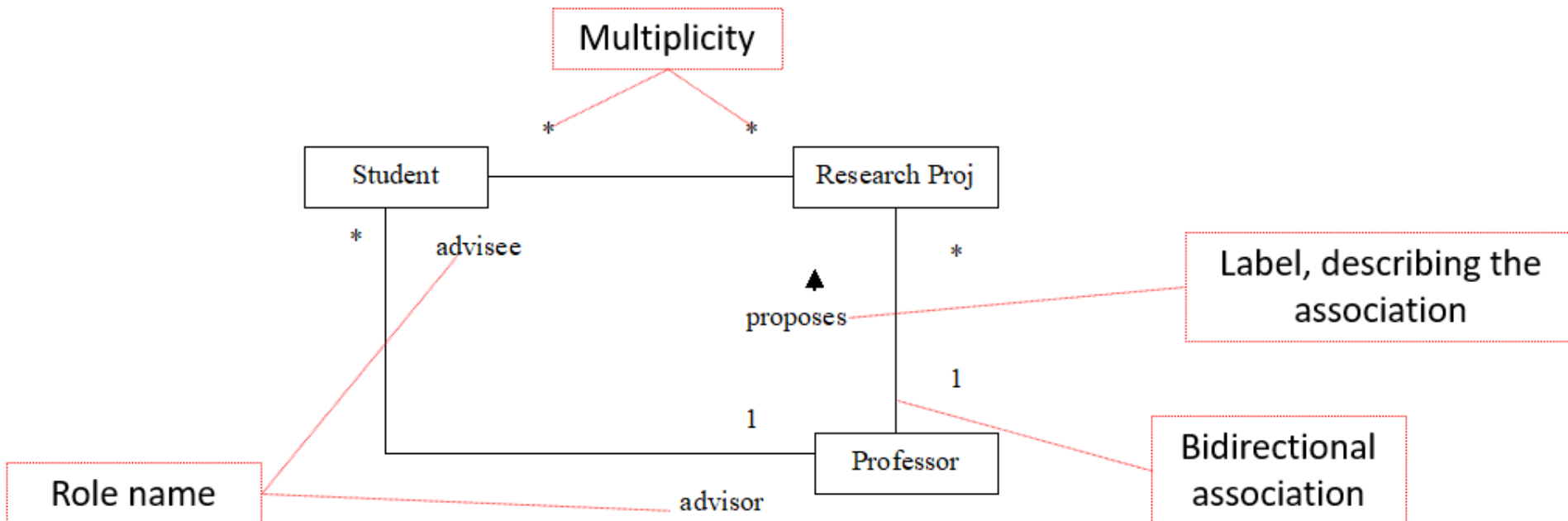


- Implementation perspective upon the system being developed

An object diagram represents a snapshot of the instances in a system and the relationships between the instances

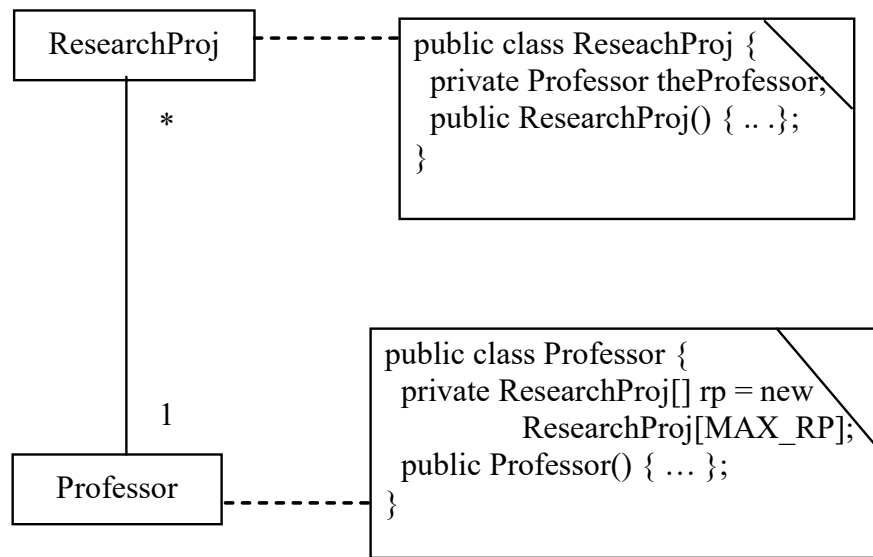
UML Modeling - Class and Object Diagrams

- Associations
 - Binary relationships among classes

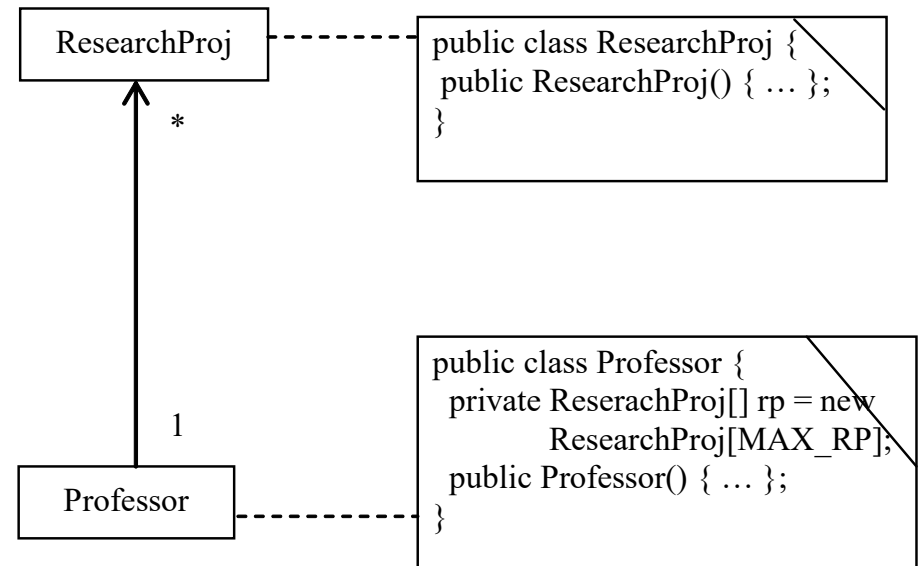


UML Modeling - Class and Object Diagrams

- Associations
 - Bi-directional vs uni-directional



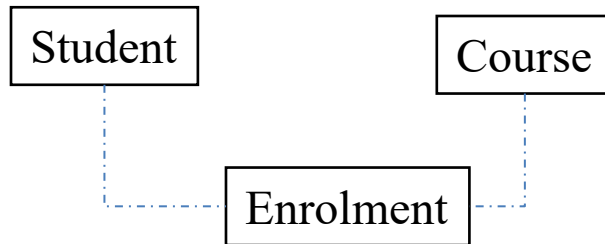
Java class skeleton



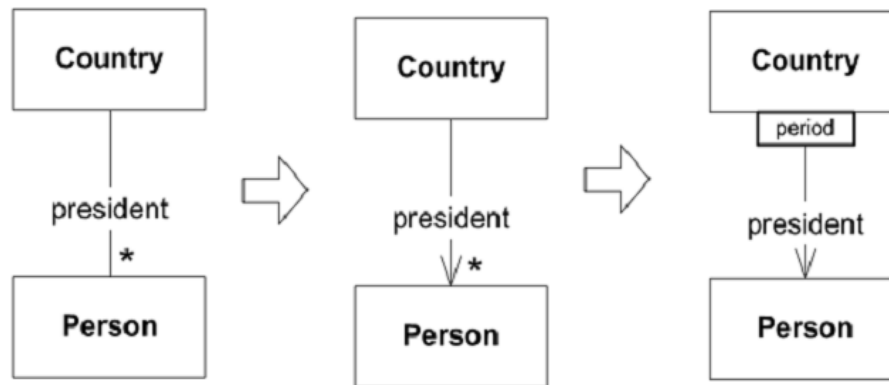
Java class skeleton

UML Modeling - Class and Object Diagrams

- Associations – Special Cases



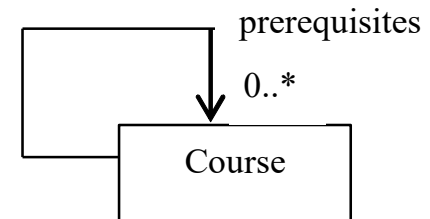
Association as a class



a) Association

b) Association traversal

c) Qualified association

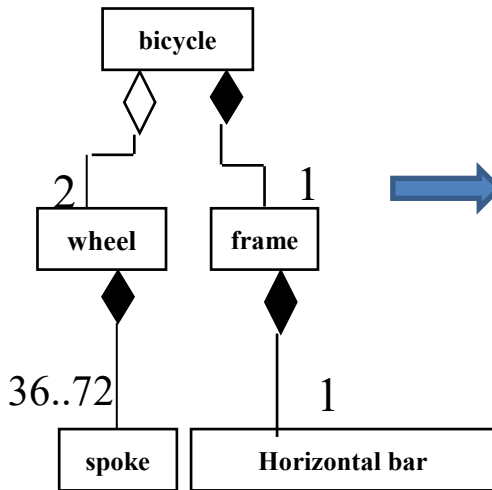


```
class Course {
    public Course() { }
    // the unbounded multiple association
    // is stored in a data structure
    private ArrayList<Course> prerequisites;
    ...
}
```

**Reflexive associations
(hierarchical or network structures)**

UML Modeling - Class and Object Diagrams

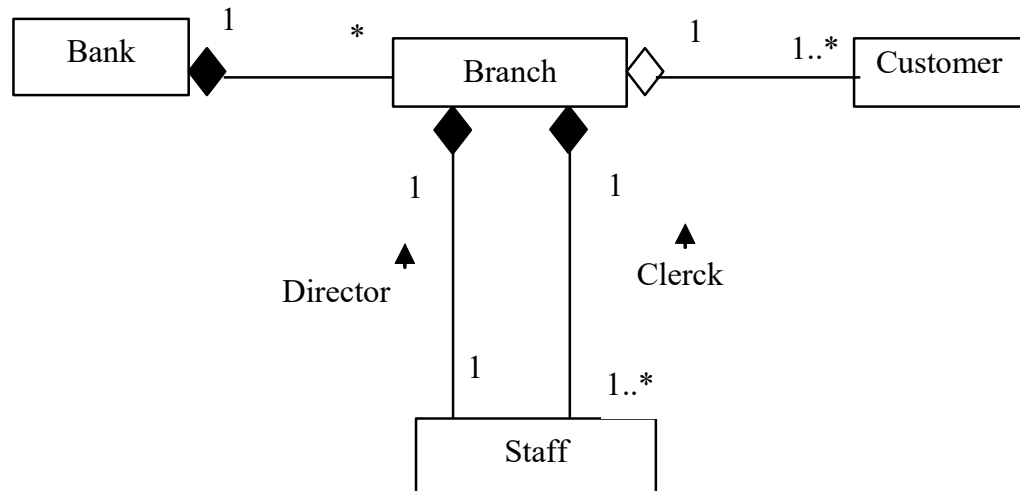
- Aggregation
 - Special form of association modeling a **“whole/part”** relationship
 - Represents a **“has – a”** or **“part – of”** relationship
 - Types: weak and strong (i.e., composition)
 - Properties: transitive, not reflexive
 - Responsibility in creating and destroying the parts



Whole	Part	Aggregation type	Motivation
Bicycle	Wheel	Weak aggregation	The bicycle and wheels can exist independently – we can add other wheels or add the wheels to another bicycle.
Bicycle	Frame	Composition	The parts have coincident lifetime with the whole – if the whole is destroyed then the parts are destroyed as well. • A frame belongs to exactly one bicycle; when you create a frame for a bicycle you must attach it to the bicycle. • Spokes belong to exactly one wheel; when you create spokes for a wheel you must attach them to the wheel. • A horizontal bar belongs to exactly one frame; when you create a horizontal bar for a frame you must attach it to the frame.
Wheel	Spoke		
Frame	Horizontal bar		

UML Modeling - Class and Object Diagrams

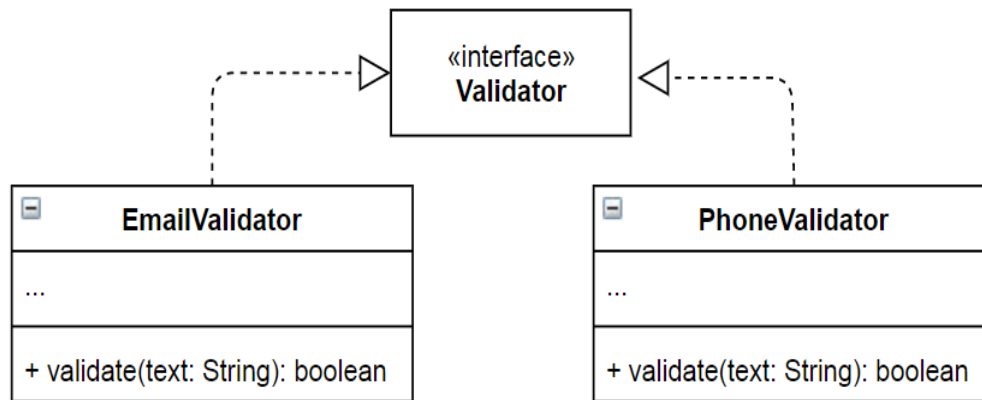
- Aggregation - Example



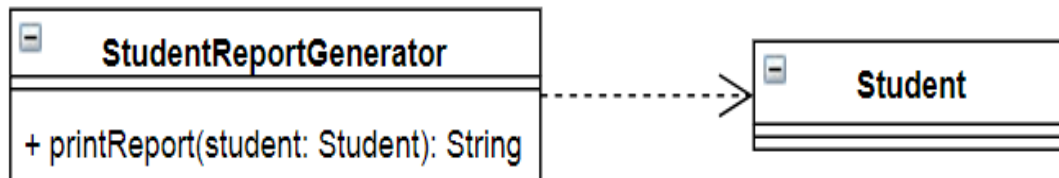
Whole	Part	Aggregation type	Motivation
Bank	Branch	Composition	The bank consists of several branches – the branches will no longer exist if the bank closes.
Branch	Staff	Composition	The staff of a branch will no longer be the employees of the branch when it will no longer exist. <i>However, the composition is debatable and depends on the interpretation: if a branch will be closed, the employees will still exist and would be assigned to another branch => we could use aggregation instead of composition.</i>
Branch	Customer	Weak aggregation	The customer of a branch will still be customers in case the branch closes and could be assigned to another branch.

UML Modeling - Class and Object Diagrams

- Realization



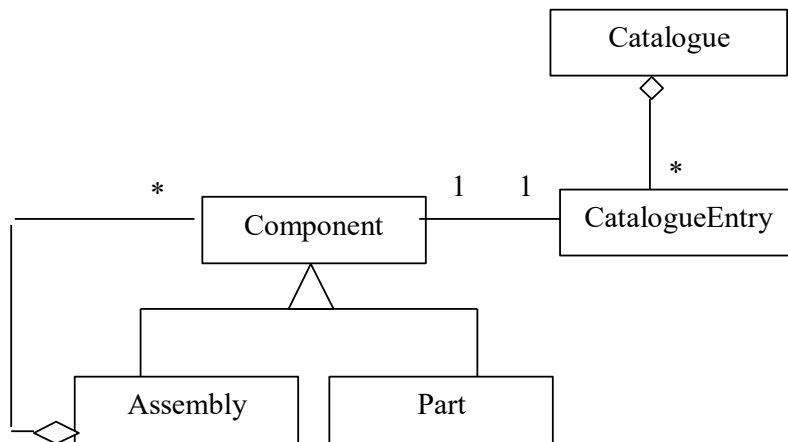
- Dependency



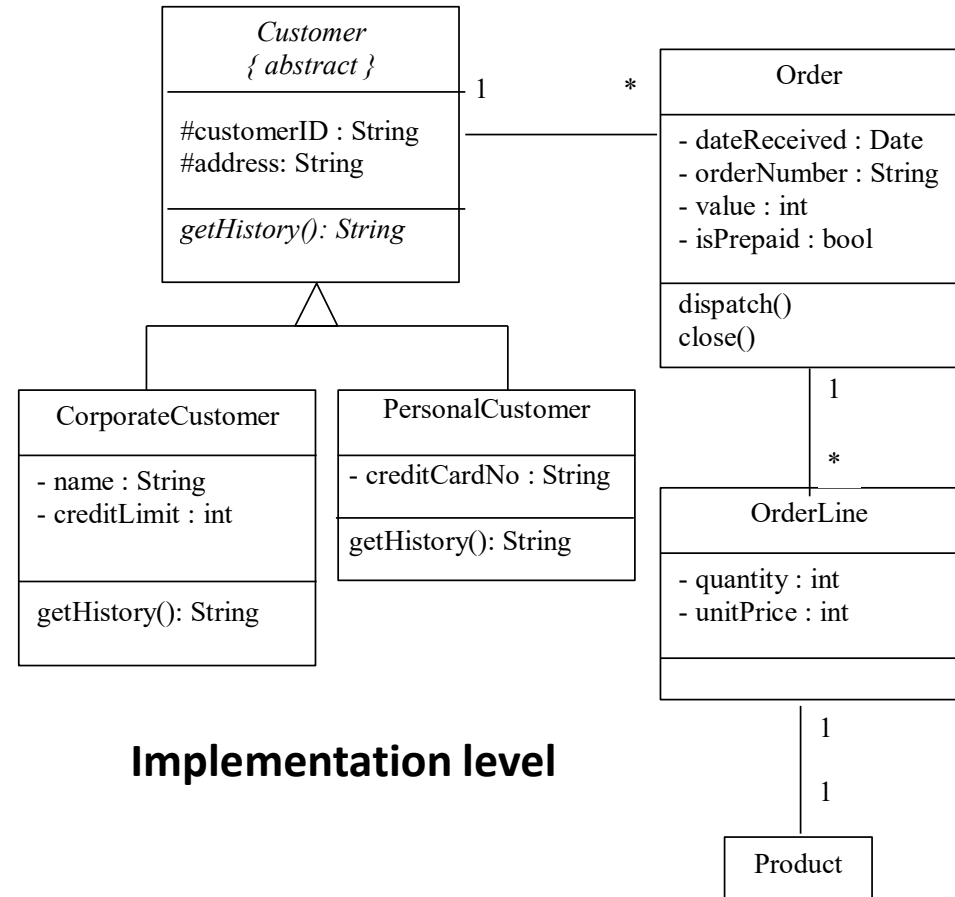
- Minimize dependencies and favor loosely coupled designs
- Improve the flexibility and maintainability of software systems.

UML Modeling - Class and Object Diagrams

- Examples



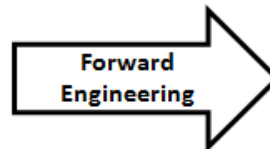
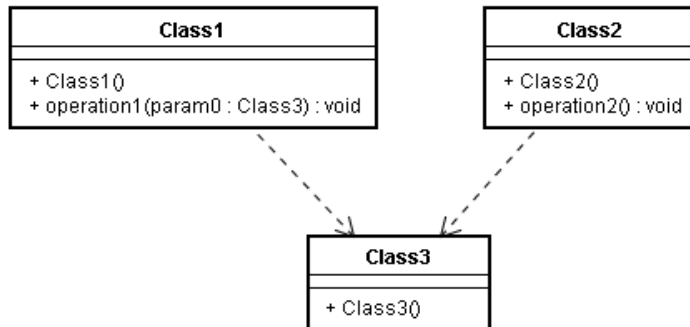
Conceptual level



Implementation level

UML Modeling - Class and Object Diagrams

- Forward Engineering - Examples



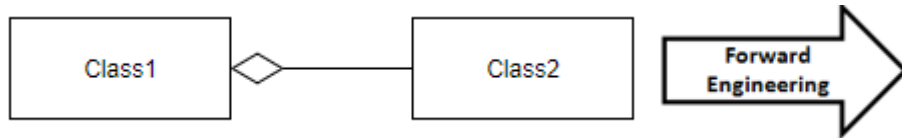
```
class Class1 {
    ...
    public Class1() {...}
    ...
    public void operation1(Class3 param0) {...}
    ...
}

public class Class2 {
    ...
    public Class2() {...}
    ...
    public void operation2()
    {
        ...
        Class3 c = new Class3();
        ...
    }
    ...
}

public class Class3{
    ...
    public Class3() {...}
    ...
}
```

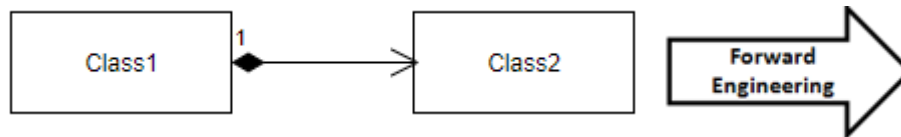
UML Modeling - Class and Object Diagrams

- Forward Engineering - Examples



```
public class Class1 {
    private Class2 class2object;
    ...
    public Class1(Class2 class2object){
        this.class2object = class2object;
    }
    ...
}
```

*Class1 is not responsible for creating the Class2 object;
The object is created outside Class1 and it is passed as an
argument to the constructor of Class1*



```
public class Class1 {
    private Class2 class2object;
    ...
    public Class1(){
        this.class2object = new Class2();
    }
    ...
}
```

*Class1 is responsible for creating the Class2 object;
The object is created when a Class1 object is created.*

Architectural Patterns

Overview

- Many developers start building applications without defining an architecture first
 - This often leads to systems referred as a big ball of mud
 - With poorly defined components
 - With tightly coupled modules
 - Difficult to modify or extend
 - With no clear system structure or direction
 - Hard to maintain over time
 - Problems Without a Defined Architecture
 - Does the system scale well?
 - What are the performance characteristics?
 - How easy is it to add new features?
 - How maintainable is the system?
 - How quickly can the system adapt to change?
- 
- The illustration depicts a massive, chaotic mound of various computer-related icons and symbols, such as hard drives, floppy disks, and circuit boards, piled haphazardly together. In the background, there's a faint blue screen displaying some code or technical information. The overall impression is one of disorganization and complexity.
- [Source 1](#)



Source 1

Architectural Patterns

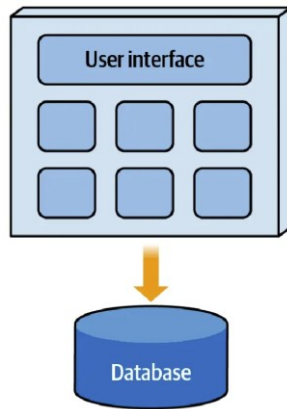
Overview

- Architectural styles
 - Help define the structure and behavior of an application
 - Different styles provide different strengths
 - Some support high scalability
 - Some make systems easier to modify
 - Some improve performance or reliability
- Choosing the right architectural style
 - requires understanding the strengths and limitations of each style and aligning them with business needs and goals
- Architectural styles versus architectural patterns
 - Architectural style – describes the macro structure of a system
 - Architectural pattern – describes a reusable structural building block pattern that can be used within each of the architectural styles to solve a particular problem

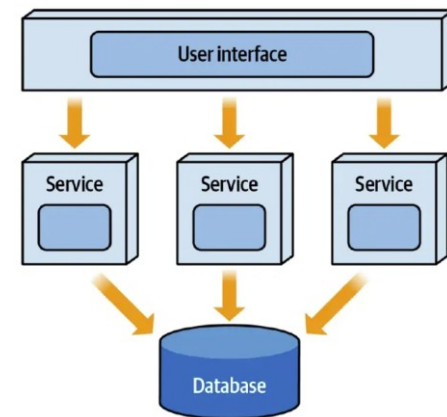
Architectural Patterns

Overview

- Architectural styles classification



Monolithic
(single deployment unit)



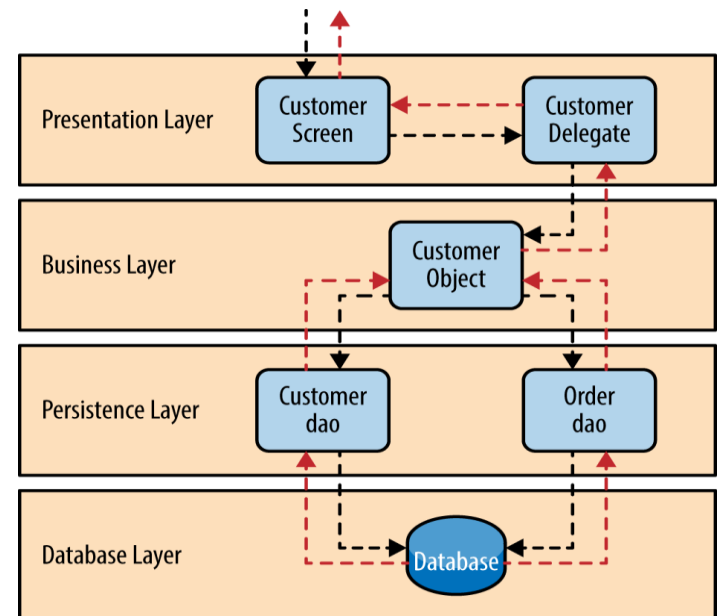
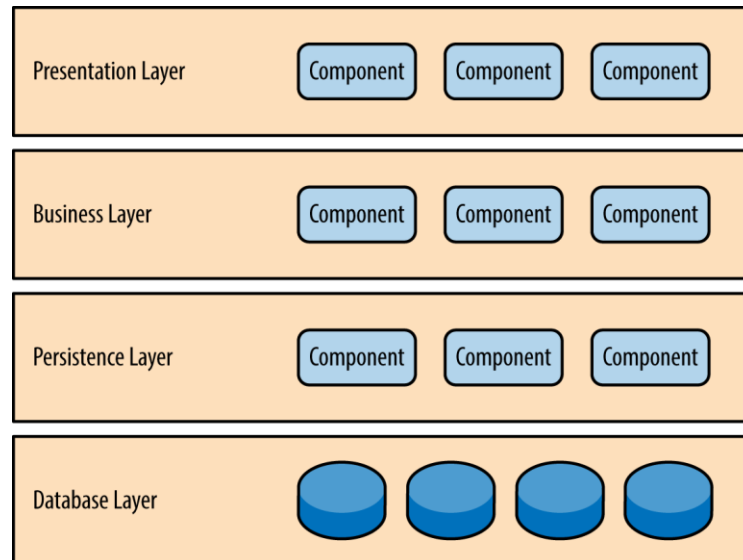
Distributed
(multiple deployment units)

- Which one to choose?
 - Monolithic style** – often used for simple applications (desktop, mobile, Web) with limited scalability requirements and developed by small teams
 - Distributed style** – for complex systems, applications requiring high scalability, systems requiring high fault tolerance, large systems with multiple architectural requirements

Architectural Patterns

Layered Architectural Pattern

- Generally considered as a monolithic architectural style
- Key features
 - Separation of Concerns
 - Layers of Isolation
 - Closed vs Open Layers
 - Standardized Communication



Architectural Patterns

Layered Architectural Pattern

- Advantages
 - Ease of development, maintainability, testability, reusability
- Disadvantages
 - Architecture sinkhole anti-pattern
 - Performance Overheads
 - Difficulty in Scaling
 - Not very fault tolerant
 - Poor Fit for Domain Driven Designs
- Suitability
 - When most changes are isolated to specific layers within the application
 - When the team structure is technically partitioned

Architectural Patterns

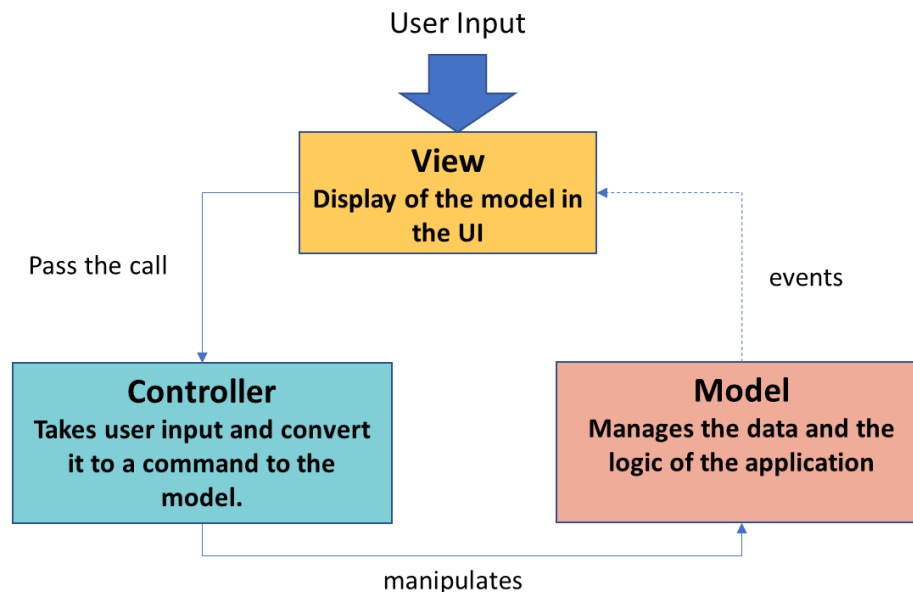
Layered Architectural Pattern

- When to avoid
 - High Scalability, Fault Tolerance, or Performance Needs
 - Complex Feature Changes
 - Cross-Functional Teams
- Use cases
 - Enterprise Applications like Customer Relationship Management
 - Web Applications like E-commerce platforms
 - Desktop Applications such as Financial Software
 - Mobile Applications like Banking applications

Architectural Patterns

Model View Controller (MVC)

- Separate an application into three interconnected components enabling modular development of each part independently
 - used to implement user interfaces, data, and controlling logic
- Key features
 - Separation of concerns
 - separation between the software's business logic and display



Architectural Patterns

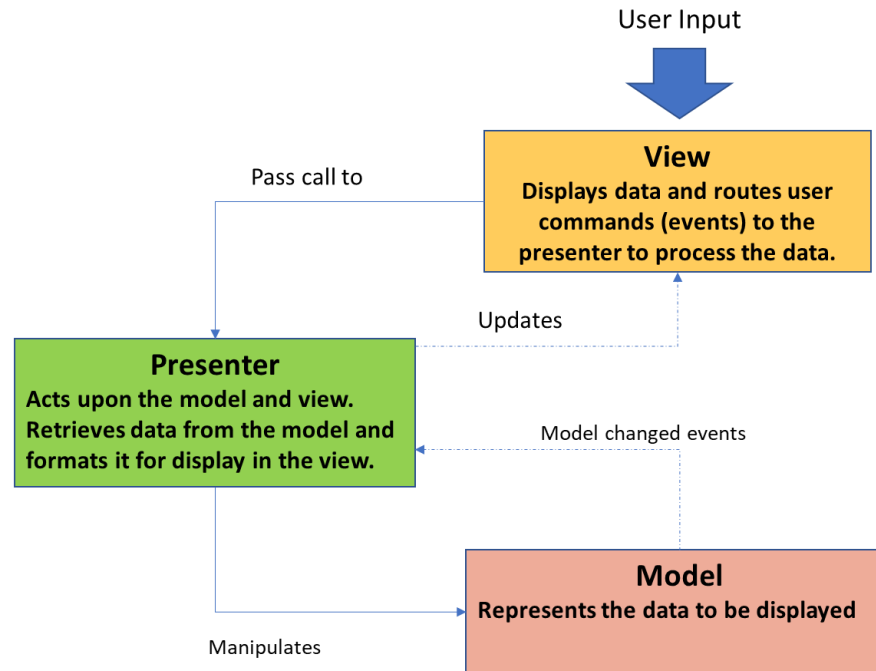
Model View Controller (MVC)

- Advantages
 - More organized and maintainable code
 - Enhances team collaboration
 - Each component can be tested independently, leading to more reliable applications and faster debugging
- Disadvantages
 - For small, simple applications, the pattern can add unnecessary complexity that slows development without providing meaningful benefits
 - can initially slow development while the team adapts to the architecture
- Use cases
 - Medium to large applications, team projects, or applications expected to grow over time
 - Data-driven applications and projects requiring multiple user interfaces
 - Web applications
 - Spring MVC, ASP.NET MVC

Architectural Patterns

Model View Presenter (MVP)

- Derivation of the MVC architectural pattern which is mostly used for building user interfaces
- All presentation logic is pushed to the presenter



[Source](#)

Architectural Patterns

Model View Presenter (MVP)

- Advantages
 - Separation of Concerns
 - Testability
 - Flexibility and Scalability
 - Reduced Dependency
 - Improved Maintainability
- Disadvantages
 - Complexity
 - Learning Curve
 - Potential Overhead
 - Tighter Coupling Between Presenter and View
 - Increased File Count
- Use cases: desktop applications, Web applications with complex user interfaces and business logic

Architectural Patterns

Model View Controller vs Model View Presenter

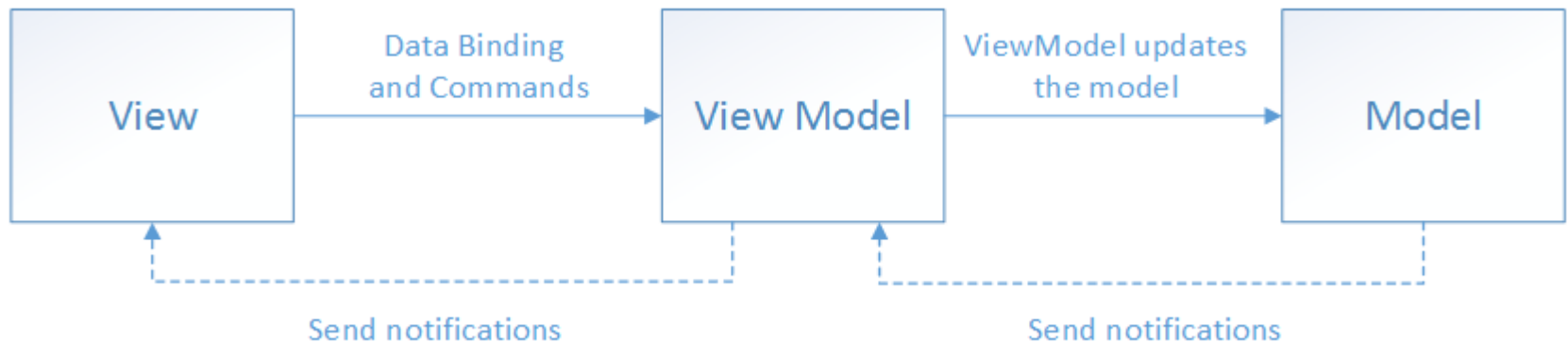
- Model View Controller
 - Controllers are behavior based and can share multiple views.
 - View can communicate directly with Model
- Model View Presenter
 - View more separated from Model. The Presenter is the mediator between Model and View.
 - Easier to create unit tests
 - Generally, there is a one-to-one mapping between View and Presenter, with the possibility to use multiple Presenters for complex Views
 - Listen to user action and model updates
 - Updates model and view as well

[Source](#)

Architectural Patterns

Model View ViewModel (MVVM)

- Helps cleanly separate an application's business and presentation logic from its user interface



[Source](#)

Architectural Patterns

Model View ViewModel (MVVM)

- Advantages

- Developers can create unit tests for the view model and the model, without using the view
- The app UI can be redesigned without touching the view model and model code
- Designers and developers can work independently and concurrently on their components during development
- Maintainability, extensibility

- Disadvantages

- For simple user interfaces, it may add unnecessary complexity
- In large applications, ViewModels can become complex
- Relies heavily on data binding
 - When bindings become complex: errors are harder to trace, bugs may appear indirectly, the UI might not update properly if bindings fail